

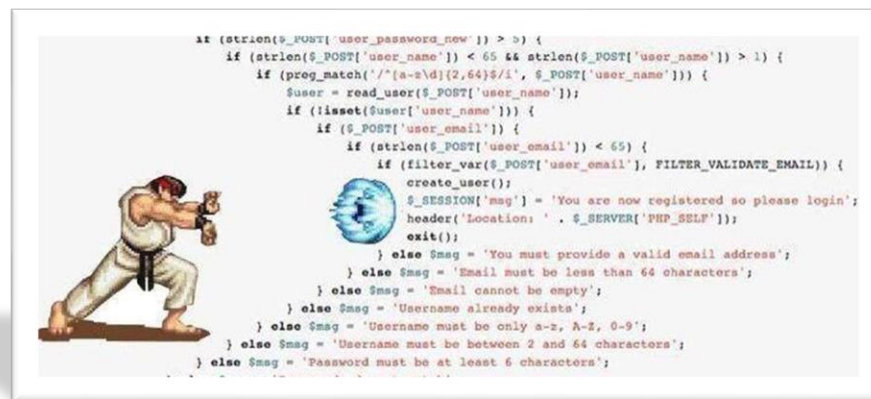
Manipulação de Strings em Python



Indentação

Antes de continuarmos e vermos um pouco mais sobre Python, precisamos entender um conceito muito utilizado em todas as linguagens de programação, e que em Python é obrigatório utilizar. Este conceito de organização das informações e do blocos adicionados ao Algoritmo é a **Indentação**.

Indentação são blocos de códigos que apresentam na sua estrutura um recuo, formando uma hierarquia em que o código será executado. Através do recuo que fazemos o alinhamento dos códigos, usa-se a **tabulação** ou **barra de espaço 4 vezes** para indicar o recuo.



Indentação

Em Python a indentação é uma das principais características da sintaxe da linguagem.

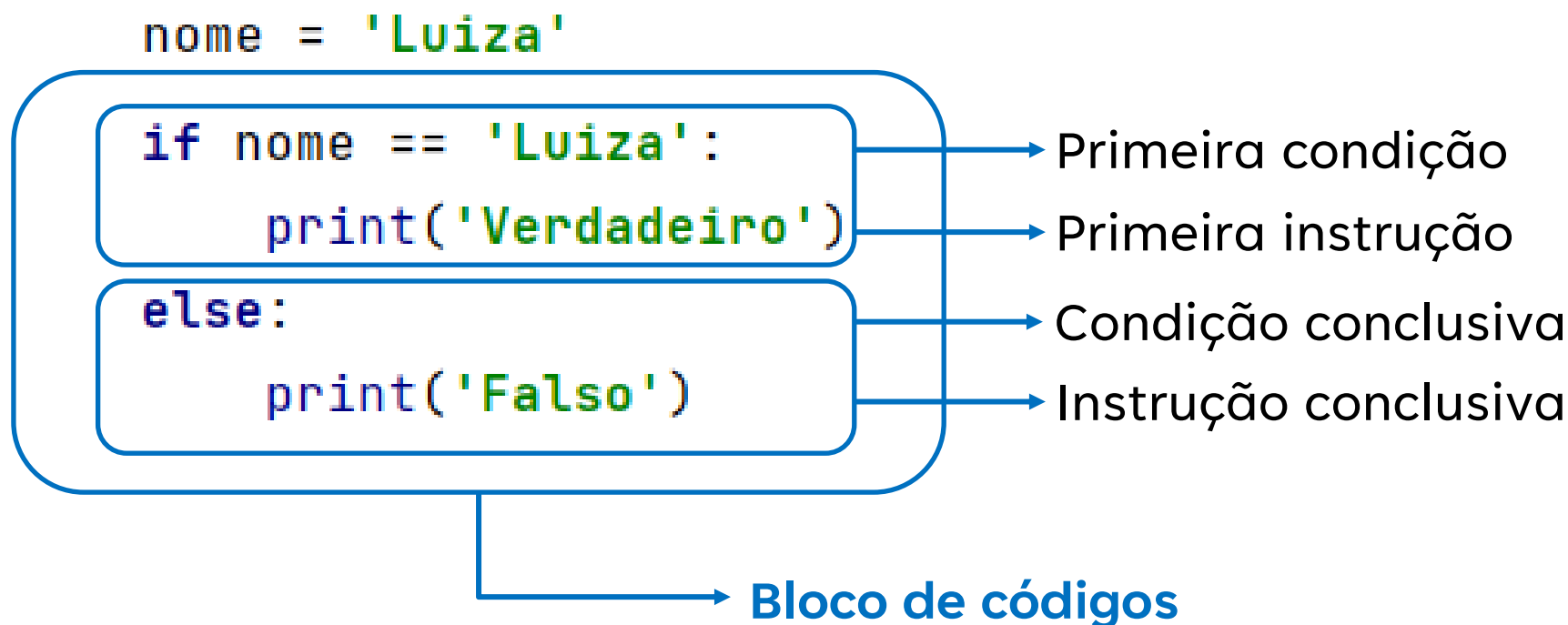
Indentação

Indentação

```
nome = 'Luiza'
if nome == 'Luiza':
    print('Verdadeiro')
else:
    print('Falso')
```

Por que a indentação é importante no Python?

A indentação é importante porque serve para indicar o início e o fim de um bloco de códigos.



A indentação deixa o código mais legível e fácil de compreender

Exemplo de código sem indentação

```
numero = 12
if numero == 15:
    print('Verdadeiro')
else:
    print('Falso')
```

No Python nem conseguiria compilar esse código, pois ele é rigoroso com indentação.

```
numero = 12
if numero == 15: print('Verdadeiro')
else: print('Falso')
```

Neste caso, o Python reconhece as instruções mesmo sem indentações. O motivo é simples, a linguagem permite a condição e instrução em uma só linha.

Declarações - Statements

No Python a continuação em uma nova linha é permitido. Através do **caracter **

```
total = 4 + \
        7 + \
        5
print(total) # 16
```

```
nome = 'maria' , \
        'Luiza' , \
        'Paulo'
print(nome)
```

Palavras-chave em Python



Palavras Reservadas na linguagem

`break` para sair de um loop

`class` define uma classe

`continue` continua para a nova iteração do loop

`def` define uma função

`del` deleta um objeto

`elif` usado em comandos condicionais, como else e if

`else` usado em comandos condicionais

`except` usado com exceções, para tratar possíveis erros

`False` Valor booleano, resulta de operações de comparação

Fonte: <https://pythoniluminado.netlify.app/sintaxe>

Palavras Reservadas na linguagem

finally	utilizado com <u>exceções</u> , um bloco de código que executará independente de ter uma exceção ou não
for	usado para criar um <u>loop</u>
from	para importar partes específicas de um <u>módulo</u>
global	declara uma <u>variável global</u>
if	usado para <u>comandos condicionais</u>
import	usado para importar <u>módulos</u>
in	capaz de checar se um valor está presente em uma lista, tupla, etc
is	testa se duas <u>variáveis</u> são iguais
lambda	cria uma <u>função anônima</u>
None	representa um valor <u>null</u>

Fonte: <https://pythoniluminado.netlify.app/sintaxe>

Palavras Reservadas na linguagem

not operador lógico de negação

or operador lógico "ou"

pass comando null, um comando que não faz nada

raise dispara um exceção

return para sair de uma função e retornar um valor

True Valor booleano, resulta de operações de comparação

try Comando de try, usado em conjunto com except

while Cria um loop while

Fonte: <https://pythoniluminado.netlify.app/sintaxe>

Strings em Python



Para strings com várias linhas, utilize 3 aspas e uma variável para guardar o valor:

Strings com várias linhas

```
texto = '''
    Este texto é de várias linhas
    Este texto é de várias linhas
    Este texto é de várias linhas
    Este texto é de várias linhas
'''

print(texto)
```

Console

Shell x

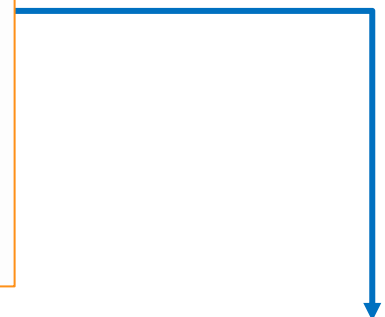
```
>>> %Run -c $EDITOR_CONTENT
```

```
Este texto é de várias linhas
Este texto é de várias linhas
Este texto é de várias linhas
Este texto é de várias linhas
```

Do jeito que você definir o texto dentro das aspas, será mostrado na tela.

```
texto = '''
    Este texto é de várias linhas
        Este texto é de várias linhas
            Este texto é de várias linhas
                Este texto é de várias linhas
    '''

print(texto)
```



```
>>> %Run -c $EDITOR_CONTENT
```

```
    Este texto é de várias linhas
        Este texto é de várias linhas
            Este texto é de várias linhas
                Este texto é de várias linhas
```

Manipulação de Strings

Strings, também chamados de cadeias de caracteres. No Python possui várias operações associadas. Vejamos algumas delas a seguir:

Saber o tamanho de uma String

```
a = 'João'  
print( len(a) )
```



len pega o tamanho da
variável

Resultado: 4

```
frase = 'Curso de Programação de Sistemas'  
print(len(frase))
```

Resultado: 32

Concatenando uma String

Juntando Strings

```
nome = input('Informe seu nome: ')\nsobrenome = input('Informe seu sobrenome: ')\nresultado = nome + sobrenome\nprint(resultado)
```

Console: JoãoPaulo



Juntando Strings



```
nome = input('Informe seu nome: ')\nsobrenome = input('Informe seu sobrenome: ')
```

```
resultado = nome + ' ' + sobrenome\nprint(resultado)
```

Console: João Paulo

String de Associação

Para verificar se uma determinada frase ou caractere está presente em uma string, podemos usar a palavra-chave **in**.

```
frase = 'Curso de Programador de Sistemas'  
print('Programador' in frase)
```



Se existe retornará
TRUE

String de Associação

Verificar se não existe
not in.

```
frase = 'Curso de Programador de Sistemas'  
print('Programador' not in frase)
```



Retornará **FALSE**, pois
existe

Modificando Strings

Python tem um conjunto de métodos integrados que você pode usar em strings.

O método **.upper()** retorna a string em maiúsculas.

```
texto = 'Texto Exemplo'  
print(texto.upper())
```

Console:
TEXTO EXEMPLO

O Método **.lower()** retorna a string em letras minúsculas.

```
texto = 'Texto Exemplo'  
print(texto.lower())
```

Console:
texto exemplo

Modificando Strings

O método **.capitalize()** retorna a primeira letra em maiúscula e o restante fica minúscula.

```
texto = 'exemplo TESTE'  
print(texto.capitalize())
```

Console:
Exemplo teste

O método **.split ("")** método retorna uma lista onde o texto entre o separador especificado se torna os itens da lista.

```
texto = 'banana uva limão, morango'  
print(texto.split(','))
```

Console:
['banana uva limão', ' morango']

Modificando Strings

O método **strip()** remove qualquer espaço em branco do início ou do fim;

lstrip remove somente a esquerda.

rstrip remove somente a direita.

```
nome = '    Maria'
print(nome.strip())
```

Console:

Maria

O método **replace('x', 'y')** substitui uma string por outra string.

```
frase = 'Texto Exemplo'
print(frase.replace('Texto', 'Teste'))
```

Console:

Teste Exemplo

MÉTODO	COMO FUNCIONA	DESCRIÇÃO
center()	print(var.center(100))	Imprime na tela com espaçamento de 100 caracteres
count(' ')	print(var.count('banana'))	Retorna o número de vezes que o valor 'banana' aparece na string
Endswith(' ')	print(var.endswith('.'))	Retorna verdadeiro ou falso se a string termina com ponto final.
isalnum()	print(var.isalnum())	Verifica se a string tem alfanuméricos, ou seja, letra do alfabeto (az) e números (0-9).
isalpha()	print(var.isalpha())	Verifique se todos os caracteres do texto são letras
isidentifier()	Print(texto.isidentifier())	Verifica se o texto digitado é uma variável válida

MÉTODO	COMO FUNCIONA	DESCRIÇÃO
islower()	print(var.islower())	Verifica se todos os caracteres do texto estão em letras minúsculas
isnumeric()	print(var.isnumeric())	Verifica se todos os caracteres são números
isupper()	print(var.isupper())	Verifique se todos os caracteres do texto estão em maiúsculas
startswith("")	print(var.startswith(""))	método retorna True se a string começar com o valor especificado, caso contrário, False.
swapcase()	txt.swapcase()	Deixe as letras que são minúsculas em maiúsculas e vice - versa

Exemplos



Usando o comando replace

```
frase = 'Curso de Programador de Sistemas'  
print(frase.replace('Programador de Sistemas', 'Python'))
```

Console

```
Curso de Python
```


Exemplos

A contagem diferencia o maiúsculo do minúsculo.

Ex.: **p** = 0 // **P** = 1 // **S** = 1 // **s** = 3

```
frase = 'Curso de Programador de Sistemas'
print(frase.count('a')) → 3
print(frase.count('A')) → 0
print(frase.upper().count('A')) → 3
```

Transformando tudo
em MAIÚSCULO e contando

Exemplo

```
info = input('Digite uma informação: ')
print('A informação é do tipo?', type(info))
print('A informação é um número?', info.isnumeric())
print('A informação é um alfabético?', info.isalpha())
print('A informação é um alfanumérico?', info.isalnum())
print('A informação é um espaço?', info.isspace())
print('A informação é capitalizada?', info.istitle())
print('A informação é maiúsculo?', info.isupper())
print('A informação é minúsculo?', info.islower())
```

Digite uma informação: Olá

A informação é do tipo? <class 'str'>

A informação é um número? False

A informação é um alfabético? True

A informação é um alfanumérico? True

A informação é um espaço? False

A informação é capitalizada? True

A informação é maiúsculo? False

A informação é minúsculo? False

Digite uma informação: 233

A informação é do tipo? <class 'str'>

A informação é um número? True

A informação é um alfabético? False

A informação é um alfanumérico? True

A informação é um espaço? False

A informação é capitalizada? False

A informação é maiúsculo? False

A informação é minúsculo? False

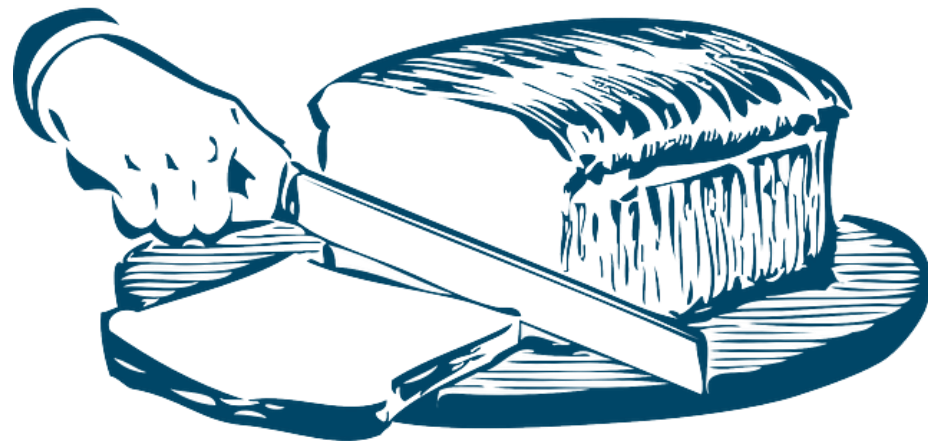
Fatiamento de Strings



Fatiamento de Strings

As Strings são iteráveis, ou seja, podemos pegar elementos específicos dentro de uma string.

Deste modo, surge o fatiamento que consiste em receber uma substring de uma determinada string, ou seja, partes específicas. Podemos definir o intervalo que desejamos receber.



Fatiamento Simples - Substrings das Strings

Podemos acessar **caracteres específicos** de uma String através de conotações []. O primeiro caractere de uma String sempre começará no índice 0.

Buscando um caractere pelo seu índice

```
caractere = 'Substrings em Python'  
print(caractere[0])  
print(caractere[5])  
print(caractere[19])  
print(caractere[20])
```

Tamanho da String: 20
Índice da String: tamanho -1

Resultado: S

Resultado: r

Resultado: n

IndexError: string index out of range

Fatiamento de sequências

Para acessar fatias de uma String, podemos especificar a posição inicial e final da fatia.

Início [0:]: É o primeiro índice a ser considerado.

Fim [:5]: É o último índice a ser considerado (sem incluir ele “5”).

`variável[início:fim]`

	0	1	2	3	4	5
$p =$	P	Y	T	H	O	N
	- 6	- 5	- 4	- 3	- 2	- 1

Exemplo - Fatiamento de sequências

p =

0	1	2	3	4	5
P	Y	T	H	O	N
- 6	- 5	- 4	- 3	- 2	- 1

`print('Tamanho:', len(p))` → Tamanho: 6 → Índice: 5

`print(p[:5])` → Começará do índice **0** até chegar no **5** - **PYTHO**

`print(p[2:6])` → Começará do índice **2** até chegar no **6** - **THON**

`print(p[0:7])` → Começará do índice **0** até chegar no **7** - **PYTHON**

`print(p[:])` → Começará do índice **0** até chegar no **último** - **PYTHON**

`print(p[-3:-1])` → Começará do índice **-3** até chegar no **-1** - **HO**

`print(p[-3:])` → Começará do índice **-3** até chegar no **último** **HON**

Exercício - Fatiamento de Strings

Considere a variável

`texto = "Programacao"`

1. Mostre as 4 primeiras letras;
2. Mostre apenas "gram";
3. Mostre as 3 últimas letras;
4. Mostre a palavra inteira usando slicing (fatiamento);
5. Mostre apenas a última letra;
6. Inverta a palavra usando slicing (fatiamento).

RESULTADO ESPERADO

```
Prog  
gram  
cao  
Programacao  
o  
oacamargorP
```


Exemplo

```
texto = "Programacao"
```

```
print(texto[:4])  
print(texto[3:7])  
print(texto[-3:])  
print(texto[:])  
print(texto[-1])  
print(texto[::-1])
```



```
Prog  
gram  
cao  
Programacao  
o  
oacamargorP
```

- `:` = separa as partes do slicing (fatiamento);
- `::` = significa que você está usando o passo;
- `-1` = andar para trás;
- `[::-1]` = inverter texto;

Aplicando CORES no python



style back

`\033[ ; ; m`

text

`\033[0;30;41maqui o texto\033[m`

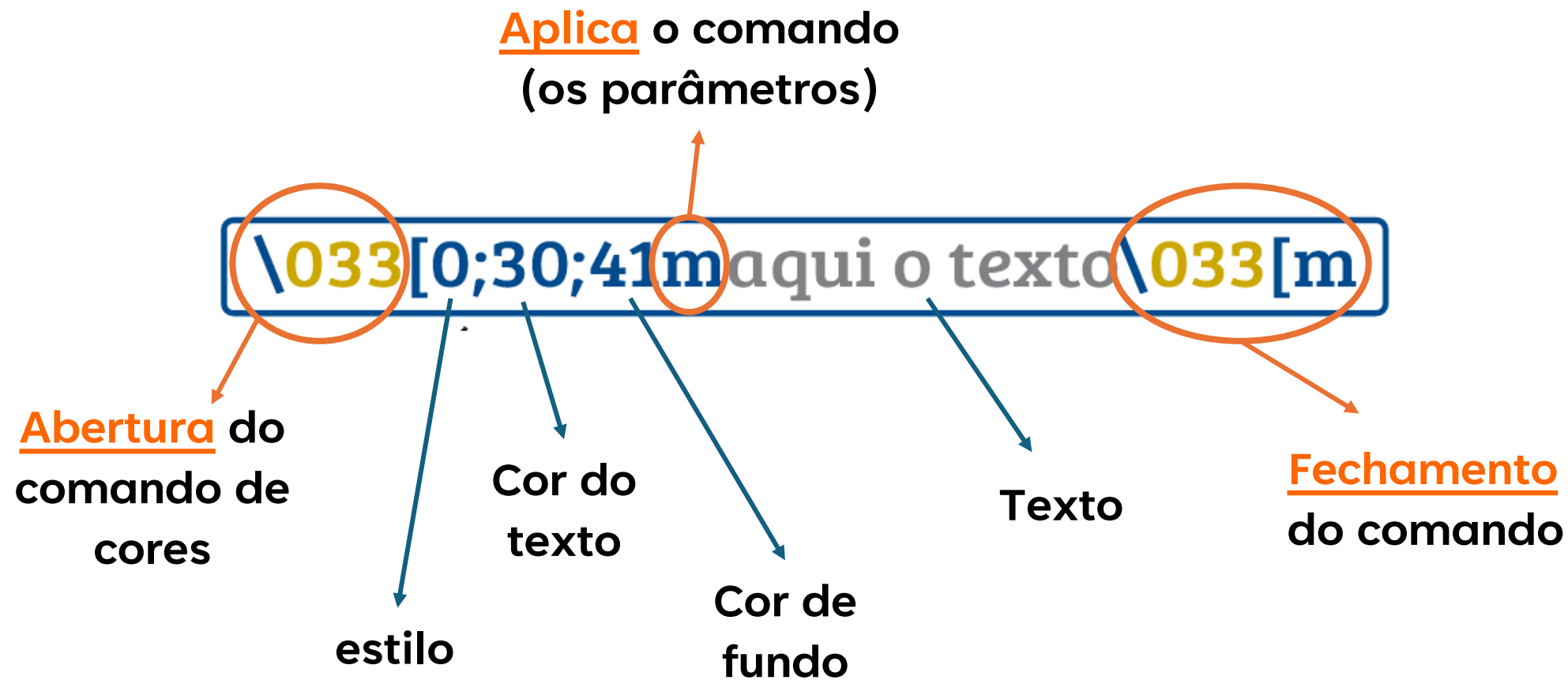
início seu texto fim

```
print('\033[0;30;41mOlá Mundo!\033[m')
```

Olá Mundo!

Tabela de Cores

		estilo	cor do texto	cor de fundo	
		style	text	back	
none (sem estilo)	0		30	40	Preto/ Branco
bold (negrito)	1		31	41	Vermelho
Underline (sublinhado)	4		32	42	Verde
Negative (inverte cor)	7		33	43	Amarelo
			34	44	Azul
			35	45	Violeta
			36	46	Azul claro
			37	47	Cinza/ Branco



```
saldo = 100  
print(f'Seu saldo é de \033[1;30;42m{saldo}\033[m reais')
```

Início

negrito

Cor do texto

Cor de fundo

Fim

Seu saldo é de 100 reais

Exemplo 1 – Chá de Revelação

```
#Chá de revelação
```

```
a = 'menino'
```

```
b = 'menina'
```

Como utilizar as 4 formas de impressão na tela:

```
print(f'0 sexo do bebê é \033[1;34m{a}\033[m ou \033[1;32m{b}\033[m?') (f-string)
```

```
print('0 sexo do bebê é \033[1;34m{}\033[m ou \033[1;32m{}\033[m?'.format(a,b))
```

```
print('0 sexo do bebê é\033[1;34m' ,a , '\033[mou\033[1;32m' ,b , '\033[m?') (vírgula)
```

```
print('0 sexo do bebê é \033[1;34m' + a + '\033[m ou \033[1;32m' + b + '\033[m?')
```

Concatenação

0 sexo do bebê é menino ou menina?

Exemplo

```
a = 'menino'
b = 'menina'

print(f'O sexo é \033[32m{a}\033[m ou \033[35m{b}\033[m?')
print(f'O sexo é \033[7;0;45m{b}\033[m!!!')
```

```
bltuç(Ł,0 sexo é /033[Ł?0?42w{p}/033[w!!!.)
```

```
O sexo é menino ou menina?
O sexo é menina!!!
```


Revisão



Exercício

Crie um programa que diga ao usuário se o número é:
PAR ou ÍMPAR

RESULTADO ESPERADO

```
Digite um número:80  
O número é Par!
```

```
Digite um número:75  
O numero é Ímpar!
```

Exemplo

```
numero = int(input('Digite um número:'))
resultado = numero % 2
if resultado == 0:
    print('O número é Par!')
else:
    print('O número é Ímpar!')
```

```
print('O número é Ímpar!')
```

```
Digite um número:80
O número é Par!
```

```
Digite um número:75
O número é Ímpar!
```

Exercício

Crie um programa que o usuário digite duas notas e apareça uma mensagem informando se ele foi:

- **APROVADO**: nota superior ou igual a 7.0
- **RECUPERAÇÃO**: nota entre 5.0 e 6.9
- **REPROVADO**: nota abaixo de 5.0

RESULTADO ESPERADO

```
Digite a primeira nota: 8
Digite a segunda nota: 9
A primeira é 8.0 e a segunda é 9.0,
a média do aluno é 8.5.
Você foi APROVADO!
```

```
Λοcε ιοτ ΑΡΒΟΛΥΔΟι
```

```
Digite a primeira nota: 6
Digite a segunda nota: 7
A primeira é 6.0 e a segunda é 7.0,
a média do aluno é 6.5.
Você esta em RECUPERAÇÃO.
```

```
Λοcε εσfε εw RECUPΕΚΥCΥO*
```

```
Digite a primeira nota: 4
Digite a segunda nota: 3
A primeira é 4.0 e a segunda é 3.0,
a média do aluno é 3.5.
Você foi REPROVADO.
```

```
Λοcε ιοτ REΠΚΟΛΥΔΟ*
```

Exemplo

```
nota1 = float(input('Digite a primeira nota: '))
nota2 = float(input('Digite a segunda nota: '))

media = (nota1 + nota2) / 2

print(f'A primeira é {nota1} e a segunda é {nota2}, a média do aluno é {media}.')

if media < 5:
    print(f'Você foi \033[7;31mREPROVADO.\033[m')

elif 7 > media >= 5:
    print(f'Você está em \033[7;33mRECUPERAÇÃO.\033[m')

elif media >= 7:
    print(f'Você foi \033[7;32mAPROVADO!\033[m')
```

```
print(f'Você foi \033[7;35mAPROVADO!\033[m')

elif media >= 7:
```

```
print(f'Você foi \033[7;32mAPROVADO!\033[m')
```

Exercício

Crie um programa que o usuário calcule o valor a ser pago na loja de acordo com as condições de pagamento:

- À vista (dinheiro/pix): 10% de desconto
- À vista (cartão): 5% de desconto
- 2x no cartão: 5% de acréscimo
- 3x até 10x no cartão: 10% de acréscimo

```
===== LOJA C&A =====
Valor das compras: R$ 500
FORMAS DE PAGAMENTO
[ 1 ] à vista dinheiro/pix (10% de desconto)
[ 2 ] à vista cartão (5% de desconto)
[ 3 ] 2x no cartão (5% de acréscimo)
[ 4 ] 3x até 10x no cartão (10% de acréscimo)
Qual a forma de pagamento? 1
O Total da compra será de R$ 450.00.
```

Desconto

$\text{valor} = \text{preco} - (\text{preco} * 10/100)$

Acréscimo

$\text{valor} = \text{preco} + (\text{preco} * 5/100)$

Exemplo

```
===== LOJA C&A =====
Valor das compras: R$ 500
FORMAS DE PAGAMENTO
[ 1 ] à vista dinheiro/pix (10% de desconto)
[ 2 ] à vista cartão (5% de desconto)
[ 3 ] 2x no cartão (5% de acréscimo)
[ 4 ] 3x até 10x no cartão (10% de acréscimo)
Qual a forma de pagamento? 1
O Total da compra será de R$ 450.00.
```

```
O total da compra será de R$ 450.00.
O valor da parcela será de R$ 450.00.
```

```
===== LOJA C&A =====
Valor das compras: R$ 500
FORMAS DE PAGAMENTO
[ 1 ] à vista dinheiro/pix (10% de desconto)
[ 2 ] à vista cartão (5% de desconto)
[ 3 ] 2x no cartão (5% de acréscimo)
[ 4 ] 3x até 10x no cartão (10% de acréscimo)
Qual a forma de pagamento? 2
O Total da compra será de R$ 475.00.
```

```
O total da compra será de R$ 475.00.
O valor da parcela será de R$ 475.00.
```

```
===== LOJA C&A =====
Valor das compras: R$ 500
FORMAS DE PAGAMENTO
[ 1 ] à vista dinheiro/pix (10% de desconto)
[ 2 ] à vista cartão (5% de desconto)
[ 3 ] 2x no cartão (5% de acréscimo)
[ 4 ] 3x até 10x no cartão (10% de acréscimo)
Qual a forma de pagamento? 3
O valor da parcela será 2x de R$ 262.50 com juros.
O Total da compra será de R$ 525.00.
```

```
O total da compra será de R$ 525.00.
O valor da parcela será de R$ 262.50.
```

```
===== LOJA C&A =====
Valor das compras: R$ 500
FORMAS DE PAGAMENTO
[ 1 ] à vista dinheiro/pix (10% de desconto)
[ 2 ] à vista cartão (5% de desconto)
[ 3 ] 2x no cartão (5% de acréscimo)
[ 4 ] 3x até 10x no cartão (10% de acréscimo)
Qual a forma de pagamento? 4
Em quantas parcelas (de 3x até 10x)? 5
O valor da parcela será 5x de R$ 110.00 com juros.
O Total da compra será de R$ 550.00.
```

```
O total da compra será de R$ 550.00.
O valor da parcela será de R$ 110.00.
```

Exemplo

```
print(f'{"=" * 20}\033[45m LOJA C&A \033[m{"=" * 20}')
```

```
preco = float(input('Valor das compras: R$ '))
```

```
print('\033[31mFORMAS DE PAGAMENTO\033[m')
print('')
\033[7;31m[ 1 ]\033[m à vista dinheiro/pix \033[32m(10% de desconto)\033[m
\033[7;31m[ 2 ]\033[m à vista cartão \033[32m(5% de desconto)\033[m
\033[7;31m[ 3 ]\033[m 2x no cartão \033[33m(5% de acréscimo)\033[m
\033[7;31m[ 4 ]\033[m 3x até 10x no cartão \033[35m(10% de acréscimo)\033[m
'')
```

```
opcao = int(input('\033[31mQual a forma de pagamento? \033[m'))
```

```
if opcao == 1:
    valor = preco - (preco * 10 / 100)
    print(f'O total da compra será de \033[7;32mR$ {valor:.2f}\033[m.')
elif opcao == 2:
    valor = preco - (preco * 5 / 100)
    print(f'O total da compra será de \033[7;32mR$ {valor:.2f}\033[m.')
elif opcao == 3:
    valor = preco + (preco * 5 / 100)
    parcela = valor / 2
    print(f'O valor da parcela será 2x de R$ \033[31m{parcela:.2f}\033[m com juros.')
    print(f'O total da compra será de \033[7;32mR$ {valor:.2f}\033[m.')
elif opcao == 4:
    valor = preco + (preco * 10 / 100)
    qualparcela = int(input('Em quantas parcelas (de 3x até 10x)? '))
    parcela = valor / qualparcela
    print(f'O valor da parcela será {qualparcela}x de R$ \033[31m{parcela:.2f}\033[m com juros.')
    print(f'O total da compra será de \033[7;32mR$ {valor:.2f}\033[m.')
else:
    print('\033[7;31mERRO\033[m\033[31m: Opção inválida! Tente novamente\033[m')
```